

## Geog 777 — Project 1 Report

October 11, 2025 - Sonja Sebree

### Project Scenario

When I began this project, I pictured myself on the GIS team at the Wisconsin Department of Natural Resources (WDNR), and asked a deceptively simple question: do higher nitrate concentrations in drinking water relate to higher cancer risk? WDNR had two key datasets in hand—recent well tests with nitrate values and cancer rates summarized by census tract—and they needed a tool that could turn those layers into answers. My goal was to build something that a colleague could open, type in a single control value, and watch the analysis unfold: a small, sturdy application that does the heavy lifting quietly but makes results obvious.

### Implementation

I built the tool in ArcGIS Pro using ArcPy for the analysis and Tkinter for the interface, showing the output from ArcGIS Pro exported map layouts. Keeping everything inside Pro's Python environment meant no extra installations for coworkers. If you have Pro, you can run the app. Early on I split the project into two scripts: one for geoprocessing and one for the GUI. That separation spared me the usual tangle of schema locks and kept the interface responsive; the GUI asks for a **K** value, calls the backend, and then just shows whatever the backend produced.

The analysis itself is a story about moving from scattered points to a shared geography. The nitrate measurements live at wells, but cancer rates live on tracts. To compare them fairly, I first needed a continuous nitrate surface. That's where **IDW interpolation** comes in: given the well points and their nitrate values, IDW estimates nitrate everywhere by letting nearer wells speak louder than distant ones. In the interface, the **K** the user types isn't a mysterious knob; it's the **power** in IDW; how sharply influence decays with distance. A higher K tightens the neighborhood, a lower K loosens the interpolation parameters a bit more.

Once I had that surface, I needed to translate it back to the tract level. **Zonal Statistics as Table** was effective, summarizing the mean nitrate for each tract. Also, this step taught me something practical: the tool reads values at cell centers, so at the default resolution some small tracts ended up with no cell centers inside them—no centers, no values. I dropped the cell size to a finer grid so at least one center landed inside every tract, and the summary stabilized. With mean nitrate now living in the same attribute table as cancer rate, I finally had the two numbers side by side in one geography.

From there I moved to modeling. I ran **Ordinary Least Squares** with cancer rate as the dependent variable and tract-level mean nitrate as the explanatory variable. OLS provides an accessible first look: are these two moving together in a way that isn't just chance? But OLS also comes with an assumption that its residuals aren't spatially autocorrelated. To test that, I turned to **Moran's I**. If the residuals clump, the model might be missing a spatial process; if they don't, the inference is on sturdier ground. Together, OLS and Moran's I gave me a quick, coherent loop: estimate, explain, then check the assumption.

The application wraps all of this into a rhythm a non-GIS colleague can follow. The left panel of the GUI asks for **K**. You press **Run Analysis** and watch the log on the right as Pro does its work—interpolating, summarizing, modeling, validating. The middle panel updates with the resulting maps: wells for context, the IDW surface for nitrate, and optional views keyed to the OLS and Moran steps. Two buttons open the full reports: a **PDF** for OLS and an **HTML** summary for Moran's I. To keep filenames predictable and file-system friendly, every output uses the entered K with periods replaced by underscores, so a run at 2.5 yields `idw_map_k2_5.png`, `OLS_2_5.pdf`, and `MoranReport_2_5.html`. It's a small detail, but it makes side-by-side comparisons of different K values pleasantly straightforward.

Under the hood, ArcPy's `arcpy.mp` module lets the backend quietly refresh layout titles and data sources inside a prebuilt Pro project, then export clean, consistent figures. The GUI doesn't fuss with any of that; it just points to whatever the backend exported most recently. That division of labor kept the codebase tidy and made debugging easier. When something failed, it was either a geoprocessing question or a display question, never both at once.

What I like about this project is the way it balances rigor and usability. The pipeline is defensible and repeatable—IDW to move points onto a common surface, zonal statistics to bring the surface down to tracts, OLS to probe the relationship, and Moran's I to keep us honest about spatial structure. But the experience is simple: type a number, click a button, read the map, open the report. That's the kind of tool a program office can live with—one that invites exploration (try  $K=2.0$ , then  $3.0$ , then  $4.0$ ) while guarding the methodological guardrails.

## Conclusion

If I had another iteration, I'd add a panel that summarizes model fit and Moran's I in plain language; just a few sentences that translate coefficients and p-values into implications for decision-making. I'd also like to capture the chosen K and results metadata into a small run log so analysts can track "what worked best where" over time. But the core is there: a compact, shareable application that turns WDNR's question into a transparent workflow and a set of maps and reports you can trust.

In the end, this project reminded me that GIS is most persuasive when it is also accessible. It's not just the map or the statistic; rather, the ease with which a colleague can ask a question, try a setting, see a result, and believe what they're seeing. This tool aims for exactly that.